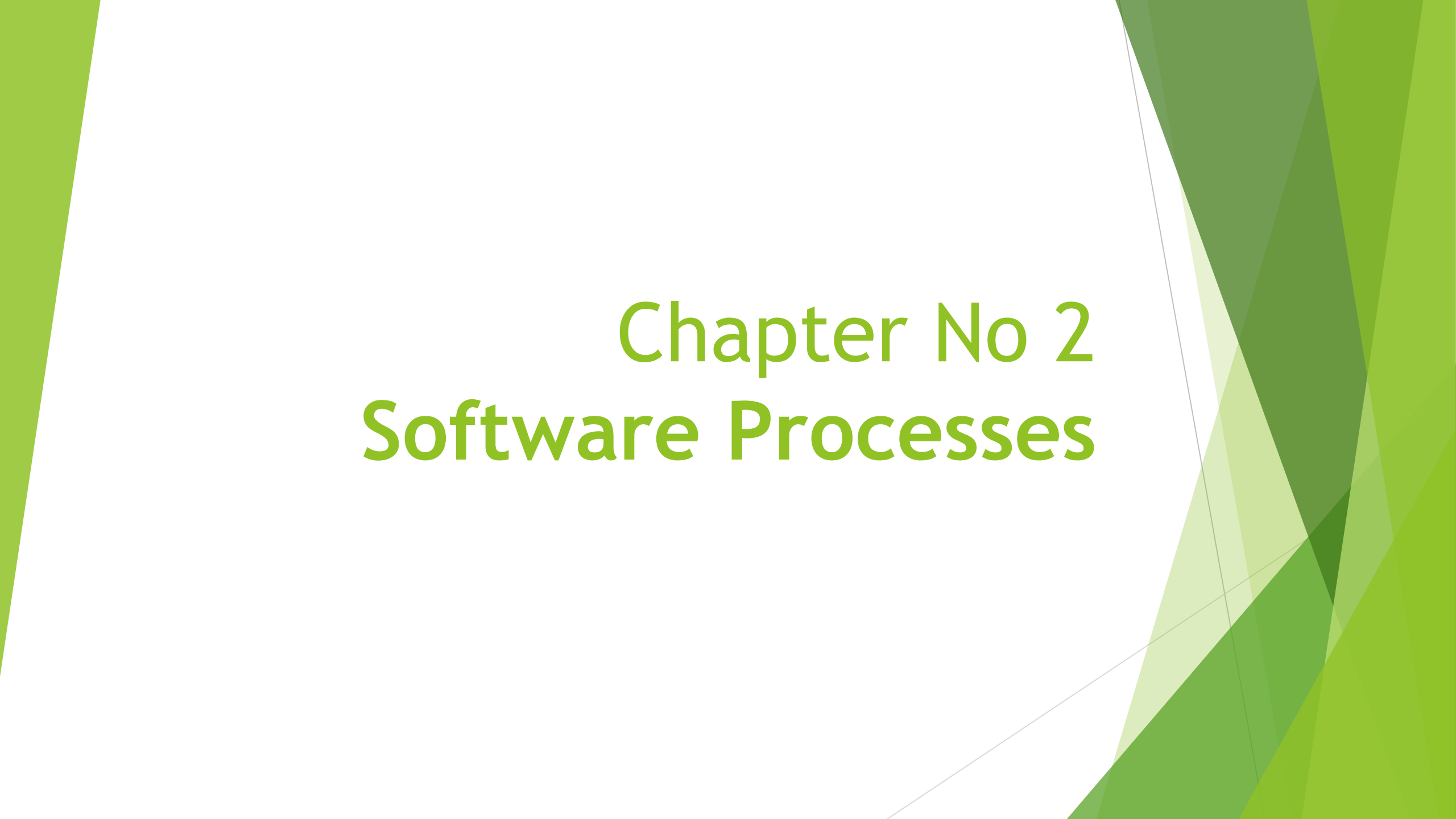


The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Chapter No 2

Software Processes

Topics covered

- ▶ Software process models
- ▶ Process activities
- ▶ Coping with change
- ▶ Process improvement

Process activities

Process Activities

Software engineering process activities are the foundational steps required to produce, deliver, and maintain software. Regardless of the methodology (like Agile or Waterfall), all software processes generally encompass four core phases: defining requirements, designing and programming, validating quality, and evolving the system over time

The Four Fundamental Activities

- **Software Specification (Requirements):** The process of understanding and defining exactly what services are required and the constraints on the system's operation. It involves requirements gathering, analysis, and documentation.
- **Software Development (Design & Implementation):** Converting the specification into an executable system. This involves architectural/component design and actual programming/coding.
- **Software Validation (Testing):** Evaluating the software to ensure it meets both functional specifications and customer expectations. This includes various levels like unit, integration, and user acceptance testing.
- **Software Evolution (Maintenance):** Modifying the software after it has been deployed to fix bugs, adapt to changing business needs, and improve performance

The Process Framework

Alternatively, the development process is often categorized into five framework activities:

Communication: Interacting with customers and stakeholders to gather requirements and understand project objectives.

Planning: Establishing work schedules, estimating budgets, assessing technical risks, and outlining the project roadmap.

Modeling: Creating architectural models, flowcharts, and design blueprints to better understand and solve the problem.

Construction: Code generation, debugging, and testing to build the functional system.

Deployment: Delivering the product (or a version of it) to the end-user for evaluation, feedback, and further refinement.

Umbrella Activities

Running concurrently alongside the framework activities are **umbrella activities**—managerial and supporting tasks necessary to keep the project on track. These typically include:

Software Configuration Management: Managing changes to system artifacts to maintain integrity.

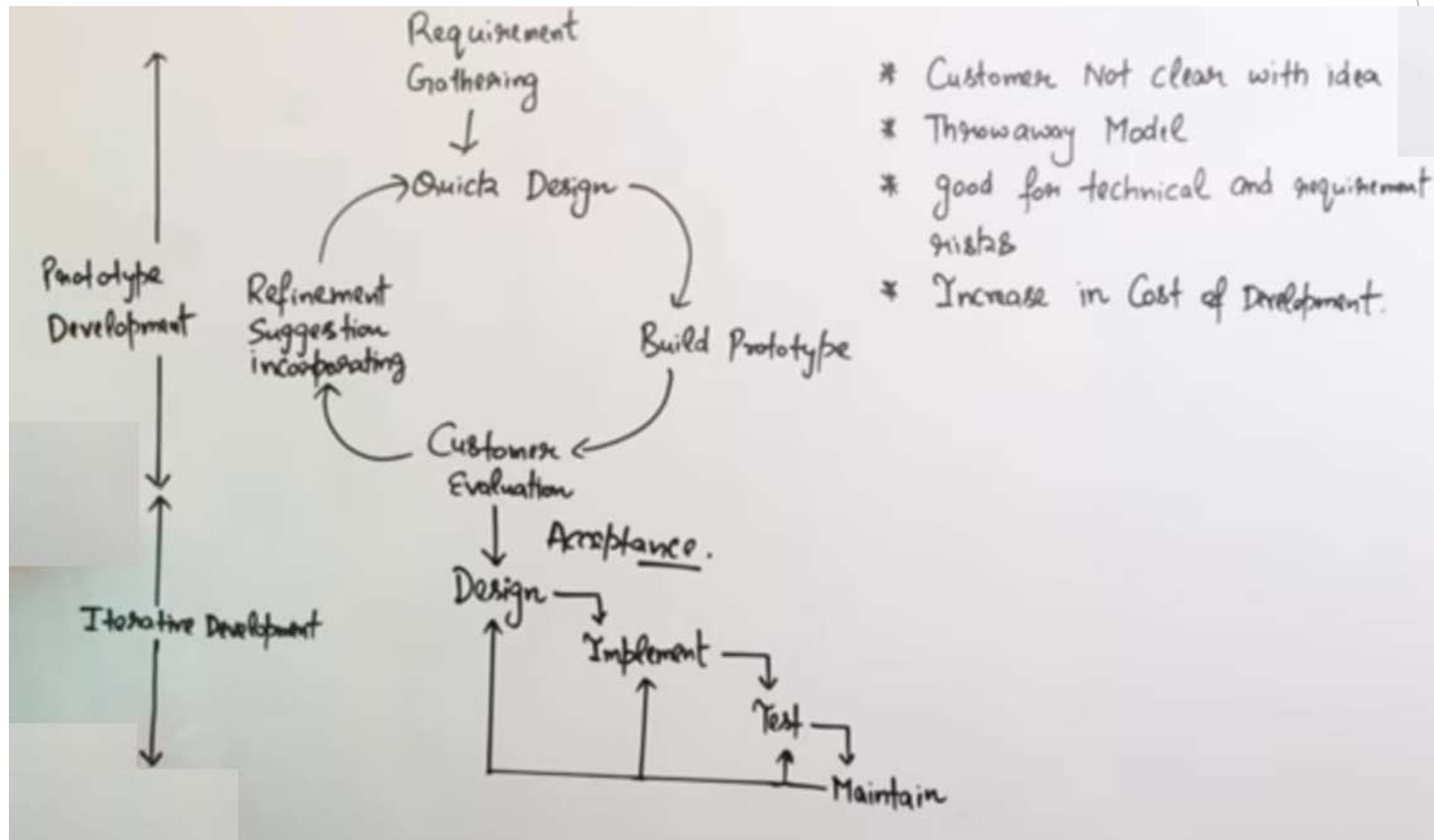
Risk Management: Identifying and mitigating potential issues that could affect quality or schedules.

Software Quality Assurance: Ensuring that the final product adheres to organizational and technical standards

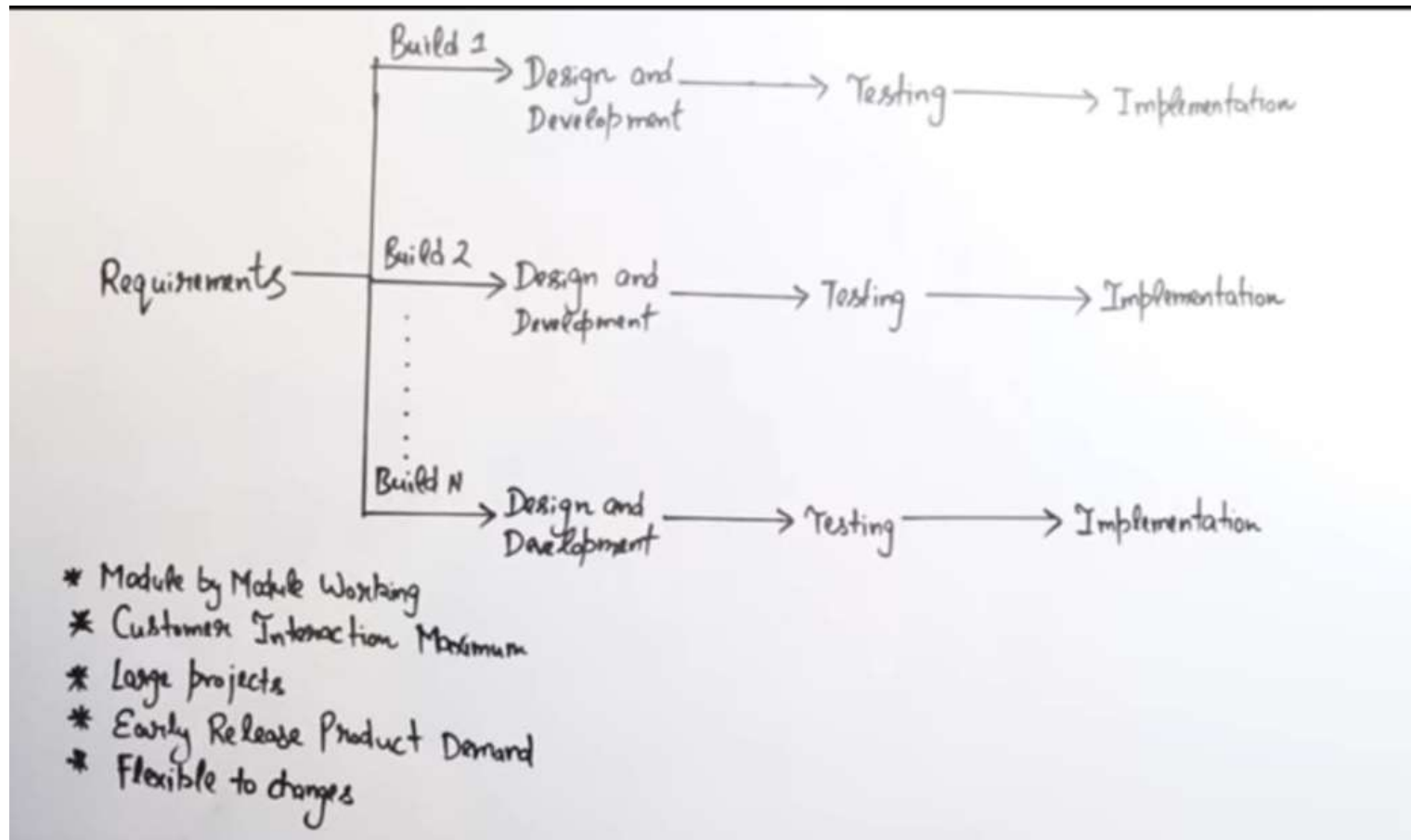
Process activities

- ▶ Real software processes are inter-leaved sequences of technical, collaborative and managerial activities with the overall goal of specifying, designing, implementing and testing a software system.
- ▶ The four basic process activities of specification, development, validation and evolution are organized differently in different development processes.
- ▶ For example, in the waterfall model, they are organized in sequence, whereas in incremental development they are interleaved.

Prototype Model



Incremental Model



Coping with change

HOW TO MANAGE CHANGE IN SOFTWARE

There are two smart ways to handle changes:

1. **Change Anticipation** – Try to guess what changes might happen and plan for them. Use tools like prototypes to test ideas early.
 2. **Change Tolerance** – Build the software in a way that allows changes without a big cost. Agile and incremental development help with this.
- **Example:** If you're making a hotel booking app, you can create a simple version first. Then based on feedback, you can add filters like "free Wi-Fi" or "pet friendly".

SOFTWARE PROTOTYPING

A prototype is a basic version of the software. It's not fully finished but shows how things will work. It helps get feedback early.



Why use a prototype?

- Saves time and cost.
 - Users can suggest improvements before full development.
- 

Example: You want to build a food delivery app. First, you show a clickable design with a menu, order page, and cart — but it doesn't work fully yet. You ask users if they like it.

COPING WITH CHANGE :

➤ Change is inevitable in all large software projects.

- *Business changes lead to new and changed system requirements*
- *New technologies open up new possibilities for improving implementations*
- *Changing platforms require application changes*

REDUCING THE COSTS OF REWORK :

- Change anticipation: where the software process includes activities that can anticipate possible changes before significant rework is required.
- Change tolerance: where the process is designed so that changes can be accommodated at relatively low cost.

Coping with Change

Coping with constant change in software engineering requires a proactive blend of agile adaptability, technical resilience, and continuous learning.

Leverage [Harness's SDLC Guide](#) to master iterative workflows, which act as the industry standard for embracing, rather than resisting, evolving requirements.

Coping with Change Cont...

Coping with change effectively breaks down into three key pillars

1. Process & Methodology

- **Incremental Delivery & Prototyping:** Avoid the trap of building a massive, rigid system. Deliver features in small, usable increments so you can get user feedback early. Validate requirements through prototypes *before* investing in full development to save on expensive rework.
- **Treat Code as Experiments:** Because business requirements shift, aim to make systems easy to alter [Why Do Developers Struggle With Small, Simple Changes?](#). Rely on modularity, abstraction, and automated testing to ensure changes don't trigger cascading failures.

Coping with Change Cont...

2. Technical Resilience & Tooling

- **Abstract the Complex:** Isolate your core logic. If a platform or third-party API changes, keeping the change localized minimizes the impact on the rest of your application.
- **Master Automated Testing:** Use automated, specification-based tests to quickly verify that your changes achieve the desired goals, rather than relying solely on manual validation.

Coping with Change Cont...

3. Personal Adaptability

- **Focus on Problem Solving:** As tools and frameworks rapidly evolve—and AI handles more of the baseline coding—your value shifts toward understanding underlying business problems and communicating effectively with stakeholders, rather than just executing code.
- **Avoid Isolation:** When facing an intimidating new technology or legacy shift, don't struggle in silence [How to Stay Ahead as a Software Engineer](#). Pair programming, peer training, and reaching out to teammates will help you learn faster and prevent burnout.

Coping with change

- ▶ Change is inevitable in all large software projects.
 - ▶ Business changes lead to new and changed system requirements
 - ▶ New technologies open up new possibilities for improving implementations
 - ▶ Changing platforms require application changes
- ▶ Change leads to rework so the costs of change include both rework (e.g. re-analysing requirements) as well as the costs of implementing new functionality

Reducing the costs of rework

- ▶ Change anticipation, where the software process includes activities that can anticipate possible changes before significant rework is required.
 - ▶ For example, a prototype system may be developed to show some key features of the system to customers.
- ▶ Change tolerance, where the process is designed so that changes can be accommodated at relatively low cost.
 - ▶ This normally involves some form of incremental development. Proposed changes may be implemented in increments that have not yet been developed. If this is impossible, then only a single increment (a small part of the system) may have be altered to incorporate the change.

Coping with changing requirements

- ▶ System prototyping, where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of design decisions. This approach supports change anticipation.
- ▶ Incremental delivery, where system increments are delivered to the customer for comment and experimentation. This supports both change avoidance and change tolerance.

Software prototyping

- ▶ A prototype is an initial version of a system used to demonstrate concepts and try out design options.
- ▶ A prototype can be used in:
 - ▶ The requirements engineering process to help with requirements elicitation and validation;
 - ▶ In design processes to explore options and develop a UI design;
 - ▶ In the testing process to run back-to-back tests.

Software Process Improvement

SPI 1. Measure and Analyze

You cannot improve what you do not measure. Development teams use metrics like cycle time, deployment frequency, and defect density to identify bottlenecks in the pipeline. By establishing a baseline, teams can track whether process changes yield an actual return on investment (ROI)

SPI 2. Frameworks and Models

Organizations often adopt structured methodologies to ensure standardizations:

- **CMMI (Capability Maturity Model Integration):** A globally recognized model that helps organizations streamline process improvement and encourage productive, efficient behaviors.
- **Agile and Scrum:** Emphasizes iterative development, continuous feedback, and sprint retrospectives to adapt workflows on a regular basis.
- **Lean Software Development:** Focuses on eliminating waste, reducing handoffs, and optimizing the flow of value to the customer.

Software Process Improvement (SPI)

3. Automation and CI/CD

A primary goal of SPI is to minimize overhead activities. Continuous Integration and Continuous Delivery (CI/CD) pipelines automate testing, building, and deployment, drastically reducing manual errors and saving engineering time.

4. Continuous Feedback loops

Regular communication and feedback are essential for a healthy engineering culture. Conducting blamemortems (post-mortems), code reviews, and pair programming allows teams to share knowledge and catch architectural or process issues early in the Software Development Life Cycle (SDLC).

The SPI Lifecycle

Most successful software process improvement efforts follow this continuous loop:

Assess: Review current workflows and identify inefficiencies.

Plan: Set concrete objectives (e.g., reduce deployment time by 20%).

Implement: Introduce new tooling, training, or updated process models.

Evaluate: Track the results using established metrics and adjust as needed.

Software Process Improvement

- Software Process Improvement (SPI) methodology is defined as a **sequence of tasks, tools, and techniques** to **plan** and **implement** improvement activities to achieve specific goals such as increasing development speed, achieving higher product quality or reducing costs.
- There is no set of ideal process models that can be used for the development or evolution of software-intensive systems and services.

Importance in Context of Team-based Development

- As software development is a largely human-based activity, building teams is an important issue.
 - ▲ Clear responsibilities can help to overcome problems with team development.
- Besides product requirements, project managers have to consider process requirements when performing projects and leading teams.
 - ▲ For example: adherence to process standards or required productivity.

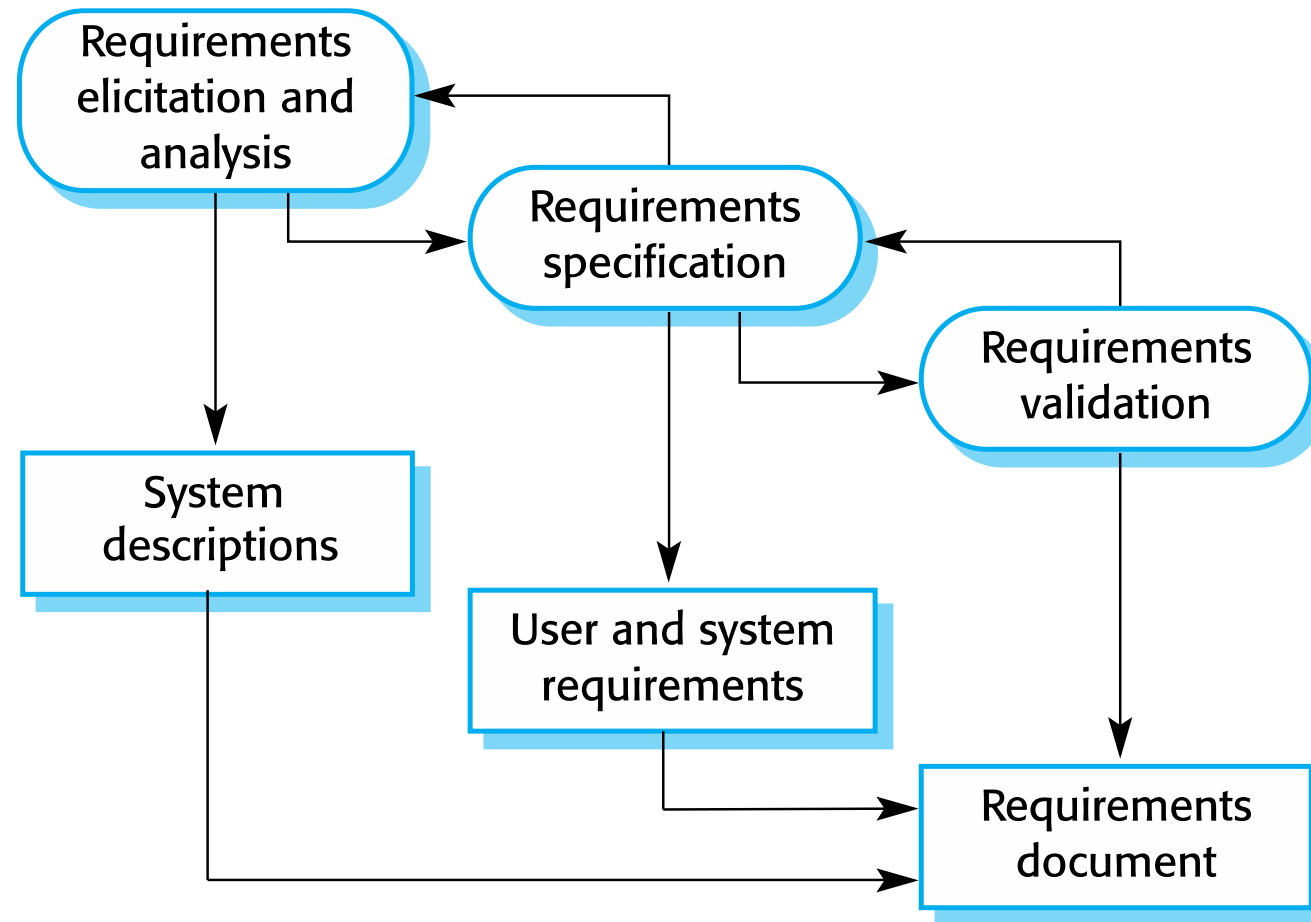
Importance in Context of Team-based Development

- Some typical characteristics are:
 - ▲ Shared activities among different developers/team members.
 - ▲ Well coordinated activities to manage the dependencies of the activities.
 - ▲ Many activities need to be synchronized so that they contribute to overall project goals

Importance in Context of Team-based Development

- Some typical characteristics are:
 - ▲ Shared activities among different developers/team members.
 - ▲ Well coordinated activities to manage the dependencies of the activities.
 - ▲ Many activities need to be synchronized so that they contribute to overall project goals

The requirements engineering process



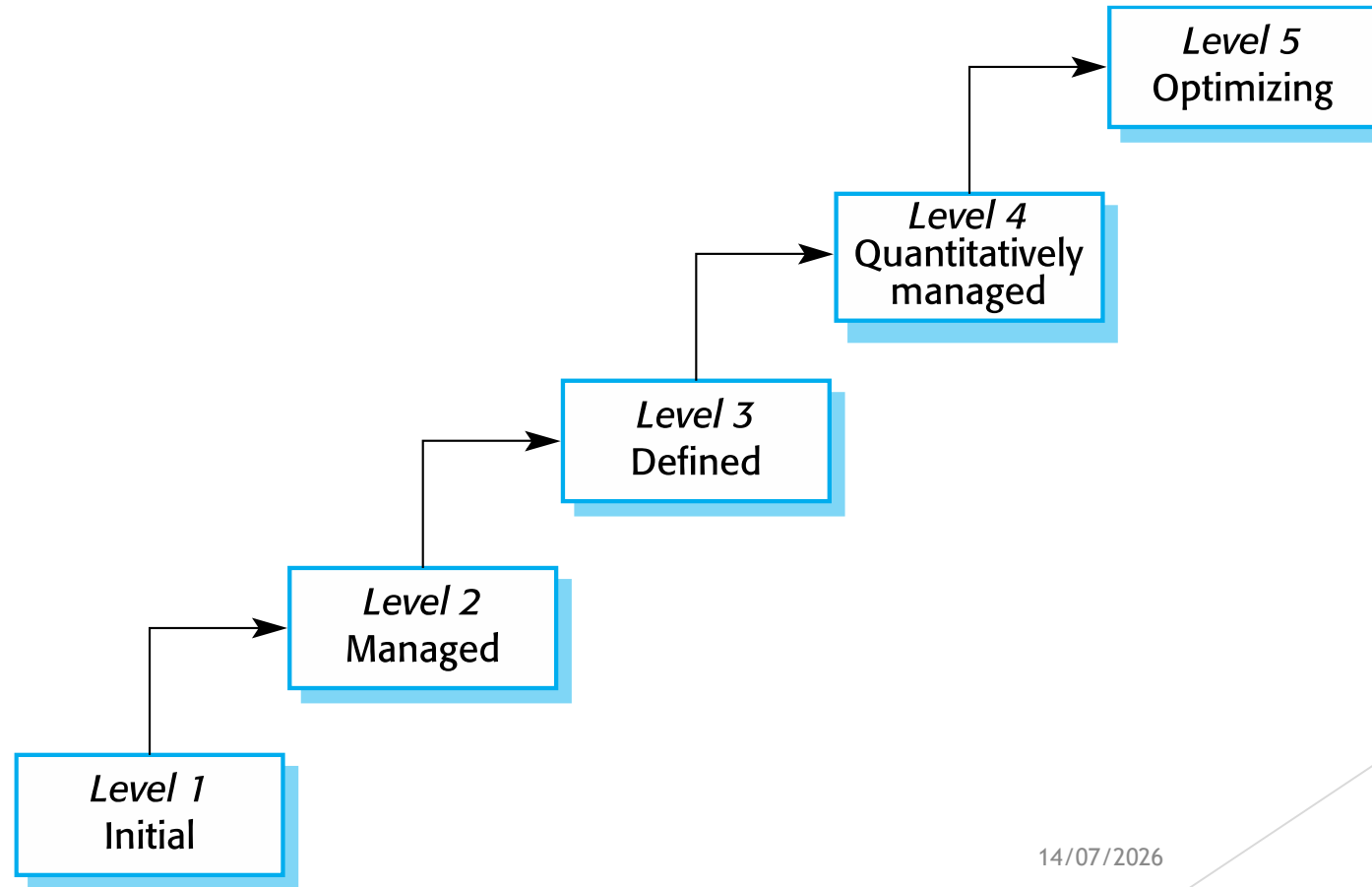
Software specification

- ▶ The process of establishing what services are required and the constraints on the system's operation and development.
- ▶ Requirements engineering process
 - ▶ Requirements elicitation and analysis
 - ▶ What do the system stakeholders require or expect from the system?
 - ▶ Requirements specification
 - ▶ Defining the requirements in detail
 - ▶ Requirements validation
 - ▶ Checking the validity of the requirements

Software design and implementation

- ▶ The process of converting the system specification into an executable system.
- ▶ Software design
 - ▶ Design a software structure that realises the specification;
- ▶ Implementation
 - ▶ Translate this structure into an executable program;
- ▶ The activities of design and implementation are closely related and may be inter-leaved.

Capability maturity levels



The SEI capability maturity model

- ▶ Initial
 - ▶ Essentially uncontrolled
- ▶ Repeatable
 - ▶ Product management procedures defined and used
- ▶ Defined
 - ▶ Process management procedures and strategies defined and used
- ▶ Managed
 - ▶ Quality management strategies defined and used
- ▶ Optimising
 - ▶ Process improvement strategies defined and used